

6. Face detection

File path:/home/pi/Yahboom_Project/1.OpenCV course/04 advanced tutorial/face_detaction.ipynb

The test code is as follows:

```
#bgr8 to jpeg format

import enum

import cv2

def bgr8_to_jpeg(value, quality=75):

    return bytes(cv2.imencode('.jpg', value)[1])
```

```
#Camera display component

import cv2

import ipywidgets.widgets as widgets

import threading

import time

import sys

image_widget = widgets.Image(format='jpeg', width=320, height=240)

display(image_widget)
```

```
image = cv2.VideoCapture(0)

image.set(3,320)

image.set(4,240)

ret, frame = image.read()

image_widget.value = bgr8_to_jpeg(frame)
```

```
# Thread function operation library

import inspect
```

```

import ctypes

def _async_raise(tid, exctype):

    """raises the exception, performs cleanup if needed"""

    tid = ctypes.c_long(tid)

    if not inspect.isclass(exctype):
        exctype = type(exctype)

    res = ctypes.pythonapi.PyThreadState_SetAsyncExc(tid,
ctypes.py_object(exctype))

    if res == 0:
        raise ValueError("invalid thread id")

    elif res != 1:
        # ""if it returns a number greater than one, you're in trouble,
        # and you should call it again with exc=NULL to revert the effect""

        ctypes.pythonapi.PyThreadState_SetAsyncExc(tid, None)

def stop_thread(thread):

    _async_raise(thread.ident, SystemExit)

```

```

# body_haar = cv2.CascadeClassifier("haarcascade_upperbody.xml")

# face_haar = cv2.CascadeClassifier("haarcascade_profileface.xml")

face_haar = cv2.CascadeClassifier("haarcascade_fullbody.xml")

#eye_haar = cv2.CascadeClassifier("haarcascade_eye.xml")

eye_haar = cv2.CascadeClassifier("haarcascade_eye_tree_eyeglasses.xml")

def Camera_display():

    while 1:

        ret, frame = image.read()

        # Convert the image to black and white

        gray_img = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

        # # Detect all pedestrians in the image

```

```

# bodies = body_haar.detectMultiScale(gray_img, 1.3, 5)

# for body_x,body_y,body_w,body_h in bodies:

# cv2.rectangle(frame, (body_x, body_y), (body_x+body_w, body_y+body_h),
# (0,255,0), 2)


    faces = face_haar.detectMultiScale(gray_img, 1.1, 3)

    for face_x,face_y,face_w,face_h in faces:

        cv2.rectangle(frame, (face_x, face_y), (face_x+face_w,
face_y+face_h), (0,255,0), 2)


    eyes = eye_haar.detectMultiScale(gray_img, 1.1, 3)

    for eye_x,eye_y,eye_w,eye_h in eyes:

        cv2.rectangle(frame, (eye_x,eye_y), (eye_x+eye_w, eye_y+eye_h),
(255,0,0), 2)


# eyes = eye_haar.detectMultiScale(gray_img, 1.3, 5)

# for eye_x,eye_y,eye_w,eye_h in eyes:

# cv2.rectangle(frame, (eye_x,eye_y), (eye_x+eye_w, eye_y+eye_h), (255,0,0), 2)


    image_widget.value = bgr8_to_jpeg(frame)

    time.sleep(0.010)

```

```

#Start thread

thread1 = threading.Thread(target=Camera_display)

thread1.setDaemon(True)

thread1.start()

```

```

#End the process and release the camera. Execute when needed.

stop_thread(thread1)

image.release()

```

After running, we can see the picture displayed by the camera below the display camera component and frame the person's face and eyes. Note that light may affect the experimental results.

```
Launcher x face_detection.ipynb +
[1]: #bgr8转jpeg格式
import enum
import cv2

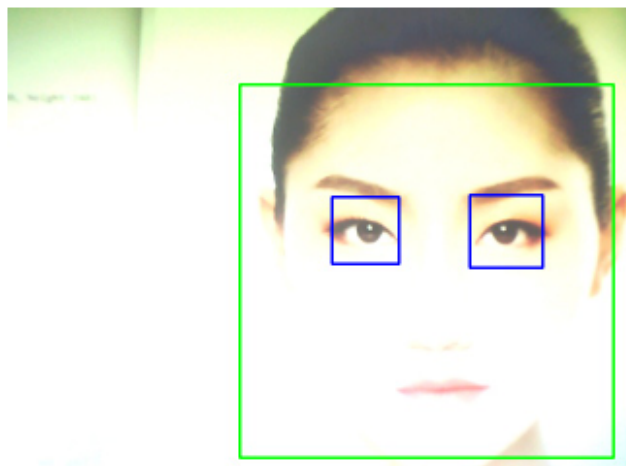
def bgr8_to_jpeg(value, quality=75):
    return bytes(cv2.imencode('.jpg', value)[1])

[2]: #摄像头显示组件
import cv2
import ipywidgets.widgets as widgets
import threading
import time
import sys

image_widget = widgets.Image(format='jpeg', width=320, height=240)
display(image_widget)

[3]: image = cv2.VideoCapture(0)
image.set(3,320)
image.set(4,240)
image.set(cv2.CAP_PROP_BRIGHTNESS, 60) #设置亮度 -64 - 64 0.0
image.set(cv2.CAP_PROP_CONTRAST, 50) #设置对比度 -64 - 64 2.0
image.set(cv2.CAP_PROP_EXPOSURE, 156) #设置曝光值 1.0 - 5000 156.0
ret, frame = image.read()
image_widget.value = bgr8_to_jpeg(frame)

[4]: # 线程功能操作库
import inspect
```



(ipykernel) | Idle

For the CSI camera, **code path:/home/pi/Yahboom_Project/1.OpenCV course/04 advanced tutorial/face_detection_CSI.ipynb**

```
#bgr8 to jpeg format
import enum
import cv2
import os

def bgr8_to_jpeg(value, quality=75):
    return bytes(cv2.imencode('.jpg', value)[1])
```

```
#Camera display component
import cv2
import ipywidgets.widgets as widgets
import threading
import time
import sys
import libcamera

from picamera2 import Picamera2
```

```
# Thread function operation library
import inspect
import ctypes
def _async_raise(tid, exctype):
    """raises the exception, performs cleanup if needed"""
    tid = ctypes.c_long(tid)
    if not inspect.isclass(exctype):
        exctype = type(exctype)
    res = ctypes.pythonapi.PyThreadState_SetAsyncExc(tid,
ctypes.py_object(exctype))
    if res == 0:
        raise ValueError("invalid thread id")
    elif res != 1:
        # ""if it returns a number greater than one, you're in trouble,
        # and you should call it again with exc=NULL to revert the effect""
        ctypes.pythonapi.PyThreadState_SetAsyncExc(tid, None)

def stop_thread(thread):
    _async_raise(thread.ident, SystemExit)
```

```
# body_haar = cv2.CascadeClassifier("haarcascade_upperbody.xml")
face_haar = cv2.CascadeClassifier("haarcascade_frontalface_default.xml")#detect
face
eye_haar = cv2.CascadeClassifier("haarcascade_eye_tree_eyeglasses.xml") #Detect
eyes wearing glasses
def Camera_display():
    while 1:
        frame = picam2.capture_array()
        # Convert image to black and white
        gray_img = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
        # Detect all pedestrians in image
        # bodies = body_haar.detectMultiScale(gray_img, 1.3, 5)
        # for body_x,body_y,body_w,body_h in bodies:
        # cv2.rectangle(frame, (body_x, body_y), (body_x+body_w,
body_y+body_h), (0,255,0), 2)
```

```
# detectMultiScale (const Mat& image, vector& objects, double scaleFactor=1.1,
int minNeighbors, int flag, cvSize)
# 1. image is the input grayscale image
# 2. objects is the rectangular frame vector group to obtain the detected object
# 3. scaleFactor is the scale parameter in each image scale, and the default
value is 1.1. The scale_factor parameter can determine how much of a jump there
is between two window scans of different sizes. Setting this parameter larger
means that the calculation will be faster, but if the window misses a face of a
certain size, objects may be lost.
# 4. The minNeighbors parameter is the number of neighbors that each cascading
rectangle should retain (failed to understand this parameter, --|||), and the
default is 3. minNeighbors controls false detections. The default value is 3,
which indicates that there are at least 3 overlapping detections before we
believe that the face does exist.
# 5. minSize is the minimum size of the target
# 6. maxSize is the maximum size of the target
```

```
faces = face_haar.detectMultiScale(gray_img, 1.1, 3)
for face_x,face_y,face_w,face_h in faces:
    cv2.rectangle(frame, (face_x, face_y), (face_x+face_w,
face_y+face_h), (0,255,0), 2)    Green wireframe outlines the detected face

eyes = eye_haar.detectMultiScale(gray_img, 1.1, 3)
for eye_x,eye_y,eye_w,eye_h in eyes:
    cv2.rectangle(frame, (eye_x,eye_y), (eye_x+eye_w, eye_y+eye_h),
(255,0,0), 2)    #Blue wireframe outlines the detected eyes wearing glasses

image_widget.value = bgr8_to_jpeg(frame)
time.sleep(0.010)
```

```
image_widget = widgets.Image(format='jpeg', width=320, height=240)
display(image_widget)
```

```
cv2.startWindowThread()
picam2 = Picamera2()
config = picam2.create_preview_configuration(main={"format": 'RGB888', "size":
(640, 480)})
#config["transform"] = libcamera.Transform(hflip=0, vflip=1)
picam2.configure(config)
picam2.start()

frame = picam2.capture_array()
image_widget.value = bgr8_to_jpeg(frame)
```

```
#Start thread
thread1 = threading.Thread(target=Camera_display)
thread1.setDaemon(True)
thread1.start()
```

```
#End the process and release the camera. Execute when needed.  
picam2.stop()  
stop_thread(thread1)
```

Note: For CSI cameras, if you want to rerun the program, you need to do the following:

